

Data Engineering Interview Preparation Guide

India Weather & Air Quality Index (AQI) ETL Pipeline | Project Defense & Knowledge Mastery

Target Role: Data Engineer / Associate DE (2026)

Focus: Airflow, Postgres DWH, MongoDB Lake, Data Quality

Author: Rishi Barapatre

Repo: github.com/RishiBarapatre/india-weather-aqi-etl

1. How to Pitch This Project (The Elevator Speeches)

Interviewers typically open with: 'Walk me through your most recent project' or 'Tell me about an ETL pipeline you built.' Do not list tools chronologically; use the **STAR method** (Situation, Task, Action, Result) focusing on engineering trade-offs.

The 60-Second High-Impact Pitch

Practice This Pitch Word-for-Word

"I built an automated, production-grade ETL pipeline that ingests live meteorological data from Open-Meteo and air quality observations from the World Air Quality Index project for 35 Indian cities on a 3-hour schedule, modeling it into a Star Schema data warehouse in PostgreSQL. The core technical challenge was handling real-world data imperfections: WAQI has no batch endpoint and experiences per-station schema drift and downtime, while legacy municipal records contain historical city names and invalid coordinates. I designed a Medallion architecture using MongoDB as a raw Bronze landing zone to preserve verbatim API payloads, orchestrated dynamic task mapping in Airflow 2.9 to isolate per-city failures, built a 3-tier entity resolution engine to reconcile city aliases, and established an automated 10-check data quality gate that logs audit metrics and halts ingestion on critical anomalies. The entire warehouse load is strictly idempotent with zero duplicate records across reruns."

The 2-Minute Deep-Dive STAR Breakdown

S - Situation	Urban Indian centers face severe seasonal air pollution and weather extremes, but environmental data is fragmented across APIs with different rate limits, missing stations, and inconsistent city naming standards.
T - Task	Build an end-to-end data platform that extracts, cleans, validates, and stores this data into an analytical warehouse while ensuring fault tolerance, idempotency, and automated data quality auditing.
A - Action	- Landed raw API JSON in MongoDB (Bronze zone) to decouple extraction from transformations. - Orchestrated dynamic task mapping in Apache Airflow for independent per-city execution and retries. - Implemented a 3-tier city resolver (exact, alias lookup, Levenshtein distance) to handle historical naming (e.g., Bombay -> Mumbai). - Modeled a Kimball Star Schema with pre-generated dim_date and PostgreSQL idempotent ON CONFLICT upserts. - Implemented a 10-point automated DQ check suite recording results to an audit table in a single transaction.
R - Result	Successfully loaded live weather and AQI facts for 34 Indian cities, processed dirty data with 100% test coverage across 77 unit tests, prevented warehouse poisoning via quarantine tables, and delivered analytical queries for rolling 24-hour averages and pollution streaks.

2. Core Technical Domains You Must Understand

Before walking into an interview, ensure you can explain the core mechanics of each domain:

Domain A: Data Architecture & Storage Layers (Medallion Pattern)

Bronze (Raw Landing Zone): MongoDB. Why? Raw JSON responses change over time. If a sensor stops reporting SO2 or WAQI uses a misspelled field (e.g. 'dominentpol'), a relational schema with strict DDL would fail during extraction. Storing documents verbatim guarantees zero data loss at ingestion and allows re-parsing historical data if transformation rules change.

Silver (Cleaning & Conformance): In-memory processing via Pandas inside Airflow workers. Standardizes timestamps to ISO-8601 UTC, sanitizes sentinel null tokens ('-', 'N/A', 999999), and resolves city foreign keys.

Gold (Curated Data Warehouse): PostgreSQL Star Schema. Facts and dimensions with primary keys, foreign keys, and analytical indexes for reporting tools and BI.

Domain B: Dimensional Modeling (Kimball Methodology)

Star Schema vs. 3NF: OLTP systems use 3NF (3rd Normal Form) to eliminate write redundancy. Data Warehouses use Star Schema (denormalized dimensions connected to central fact tables) to optimize read performance, minimize joins, and make schemas intuitive for analysts.

Surrogate Keys vs. Natural Keys: We use auto-incrementing integer surrogate keys (`city\_id`, `date\_key`, `reading\_id`) instead of natural keys (like city name strings or timestamps). Surrogate keys decouple the warehouse from source system changes, protect against natural key mutations, and offer superior join performance in B-tree indexes.

Role of the Date Dimension (`dim\_date`): Pre-populating 4,000+ dates (2020-2030) with pre-calculated calendar attributes (`year`, `quarter`, `week\_of\_year`, `is\_weekend`) eliminates runtime CPU-heavy `EXTRACT()` and `DATE\_TRUNC()` operations and enables outer joins to detect days where stations failed to record observations.

SCD Concepts: `dim\_city` currently implements SCD Type 1 (overwrite). In an interview, explain that SCD Type 2 would introduce `effective\_date`, `end\_date`, and `is\_current` columns to track historical changes (e.g., city population or tier promotions over time).

Domain C: Airflow Orchestration & Pipeline Resilience

Dynamic Task Mapping: Feature introduced in Airflow 2.3 (`expand()`). Allows the number of tasks to be determined at runtime based on the upstream output of `get\_active\_cities`. Essential for per-station API calls because each city becomes its own task instance with its own retry state and timeout.

Idempotency: An ETL pipeline is idempotent if executing it multiple times with the same input produces the exact same state as executing it once. We achieve this in MongoDB via upserts on compound key `(source, city\_id, run\_id)` and in PostgreSQL via `ON CONFLICT (city\_id, reading\_ts) DO UPDATE`.

Failure Isolation: Transient network errors trigger Airflow task retries with exponential backoff (`retries=3, retry\_delay=2m`). Non-transient errors (dead stations) are caught in Python and logged as structured failure dictionaries rather than bubbling unhandled exceptions, allowing the remaining 33 cities to complete successfully.

Domain D: Advanced SQL Techniques

Window Functions vs. GROUP BY: `GROUP BY` collapses rows; window functions (`OVER (...)`) compute aggregate values while preserving individual row identity.

`RANGE` vs. `ROWS` Frame: `ROWS BETWEEN 24 PRECEDING AND CURRENT ROW` counts physical rows (if readings are missing, you average across days). `RANGE BETWEEN INTERVAL '24 hours' PRECEDING AND CURRENT ROW` filters strictly by the timestamp value in the ordering column, handling variable observation intervals correctly.

Gaps-and-Islands (`rn - rn2`): A classic SQL interview challenge. Row number across all rows (`rn`) minus row number across qualifying rows (`rn2`) produces an invariant constant that remains identical throughout an unbroken consecutive streak, then jumps when a gap occurs.

3. Defending Architectural Choices (The 'Why' Questions)

Interviewers will probe your architectural justification. Never say *'because the tutorial said so'*. Use these explicit trade-off arguments:

Q1: Why did you use MongoDB for raw ingestion instead of staging tables in PostgreSQL?

Core Defense: *Decoupling ingestion from schema enforcement protects against external API drift.*

- Third-party APIs are external dependencies outside our control. If WAQI adds new pollutant fields or changes nested structure, a relational table with fixed columns would require DDL schema migrations or fail ingestion.
- MongoDB stores the raw payload verbatim (including API response headers and execution metadata). If a downstream business rule is modified 6 months later, the original source data is preserved in Bronze to re-run historical transforms without calling the API again.
- Separates ingestion latency from transactional warehouse locking.

Q2: Why did you run two separate PostgreSQL containers instead of one database with multiple schemas?

Core Defense: *Resource isolation and operational independence between workflow metadata and analytical workload.*

- Airflow's internal database stores task heartbeats, execution states, and connection credentials. It experiences high-frequency concurrent writes from the scheduler and workers.
- The Analytical Data Warehouse (`warehouse-postgres`) stores dimensional tables and fact data subjected to heavy analytical queries and window aggregates.
- Coupling both onto the same database instance is a known anti-pattern: an unoptimized analytical query in your DWH could lock system resources or max out CPU, causing the Airflow scheduler to drop heartbeats and mark running tasks as dead.

Q3: Why use Airflow Dynamic Task Mapping instead of a Python `for` loop inside a single task?

Core Defense: *Granular task observability, concurrency control, and blast-radius containment.*

- In a single task with a loop, if city #30 of 35 fails or times out, the entire task fails. You would have to re-fetch cities 1 to 29 on retry, wasting network bandwidth and risking API rate limits.
- Dynamic Task Mapping (`expand(city=cities)`) creates 34 discrete Airflow TaskInstances. Each city is retried independently.
- Airflow UI gives instantaneous visual feedback on exactly which city station failed, and tasks can execute concurrently across worker threads.

Q4: Why batch Open-Meteo in groups of 10 instead of all 35 at once or 1 by 1?

Core Defense: *Balanced network payload vs. failure blast radius and URL length safety.*

- Open-Meteo allows comma-separated latitude and longitude pairs. Ingesting one-by-one causes 35 separate HTTP handshakes, creating socket overhead and latency.
- Passing all 35+ coordinates in a single GET query approaches HTTP URL length thresholds (2,048 characters) and creates an 'all-or-nothing' failure risk.
- Batching in chunks of 10 gives the optimal balance: only 4 network calls total, short URL strings, and if one batch fails, 75% of cities still ingest successfully.

Q5: Why build custom Python Data Quality checks instead of using Great Expectations or dbt test?

Core Defense: *Zero framework overhead, direct transaction binding, and complete test transparency.*

- Great Expectations is a heavy dependency requiring extensive JSON context configs and multi-megabyte artifact storage. In an entry/junior role, interviewers want to see that you can write SQL and Python validation logic yourself.
- Our custom runner executes all 10 checks within the same transaction that writes to `pipeline_run_log` and `dq_check_results`.
- Plain Python functions are easily unit-tested with standard `unittest.mock` (as demonstrated by our 24 passing unit tests in `test_dq_checks.py`) without running external engines.

Q6: How does your pipeline ensure true idempotency?**Core Defense:** *Deterministic natural key constraints coupled with SQL upsert semantics.*

- Both `fact\_weather\_reading` and `fact\_air\_quality\_reading` feature a compound UNIQUE constraint on `(city\_id, reading\_ts)`.
- The loader executes `INSERT INTO fact\_... VALUES (...) ON CONFLICT (city\_id, reading\_ts) DO UPDATE SET temperature\_c = EXCLUDED.temperature\_c, ...`.
- If Airflow re-triggers a task due to a worker restart or backfill, existing records are updated in place rather than generating duplicate rows.

4. High-Probability Interview Questions & Model Answers

Study these questions categorized by the core competencies recruiters evaluate:

Category 1: API Ingestion & Network Resilience

Q7: What happens if the WAQI API returns a 500 error or times out for Delhi?**Core Defense:** *Layered resilience: Tenacity retry inside Airflow task -> structured failure catch.*

- First, the `@retry\_on\_http\_error` decorator (powered by Tenacity) performs 3 exponential backoff retries with jitter directly at the HTTP request layer.
- If all 3 attempts fail, the exception is caught inside `\_fetch\_aqi()`. The task logs the error and returns `{ 'city\_id': 1, 'status': 'error', 'stored': False }`.
- Airflow records that individual mapped task as handled, increments the `cities\_failed` counter in `pipeline\_run\_log`, and continues processing the remaining 33 cities without terminating the entire pipeline run.

Q8: How do you handle API rate limits if you scale from 35 cities to 1,000 cities?**Core Defense:** *Architectural evolution: Connection pooling, rate limit throttling, and worker pools.*

- In Airflow, configure an Airflow Pool (`waqi\_api\_pool`) with limited concurrency slots (e.g., 5 parallel tasks) to prevent flooding the endpoint.
- Implement token-bucket rate limiting or Redis-backed distributed semaphores.
- For large enterprise scale, decouple extraction into an asynchronous message queue (e.g., AWS SQS or Kafka) where worker nodes consume at a controlled rate matching the API provider's tier.

Category 2: Data Cleaning & Entity Resolution

Q9: Walk me through how your pipeline handles a messy city name like 'Bombay' or 'bengaluru'.**Core Defense:** *Three-tier deterministic-to-probabilistic resolution pipeline.*

- Step 1 (Normalization): Strip whitespace and normalize case to Title Case ('Bombay', 'Bengaluru').
- Step 2 (Exact Alias Match): Query `dim\_city\_alias`. 'Bombay' matches the raw alias for canonical city 'Mumbai' (`canonical\_city\_id = 14`). It resolves in O(1) time via B-tree index.
- Step 3 (Fuzzy Match): If unknown, RapidFuzz computes the token-sort ratio against `dim\_city`. If score ≥ 90 , the match is accepted and automatically inserted into `dim\_city\_alias` with `match\_method='fuzzy'` so future runs resolve instantly.
- Step 4 (Quarantine): If score < 90 , the raw name is written to `quarantine\_unmatched\_city\_names` and the record is dropped for this cycle to prevent corrupting analytics.

Q10: Why not just use fuzzy matching on every record every time?**Core Defense:** *Performance optimization and defense against false-positive string collisions.*

- Fuzzy matching (Levenshtein distance) is CPU-intensive ($O(N \times M)$ string comparison). Running it on thousands of rows per run would degrade pipeline throughput.
- The alias table acts as a persistent lookup cache. Once resolved, subsequent runs execute via indexed hash/B-tree lookups.
- Prevents false positives on geographically close names (e.g., 'Kalyan' vs 'Kalyani').

Category 3: Data Quality & Governance

Q11: What is the difference between a WARN check and a FAIL check in your DQ runner?

Core Defense: *Severity classification based on physical impossibility vs. transient statistical anomaly.*

- FAIL Checks (Blockers): Indicate corrupt or impossible data. Examples: Humidity < 0% or > 100%, NULL primary keys, missing schema columns, duplicate natural keys. A failure raises `RuntimeError`, turning the Airflow task red and stopping downstream processes.
- WARN Checks (Alerts): Indicate potential environmental extremes or partial missingness that should be audited but shouldn't halt the pipeline. Examples: Temperature > 50°C (could be a legitimate heatwave in Rajasthan), AQI > 400 (severe smog event in Delhi), or 15% of stations offline.

Q12: Where do you store data quality metrics, and how would you query them?

Core Defense: *Audited in `dq\_check\_results` and `pipeline\_run\_log`.*

- Every execution writes to `dq\_check\_results` with `check\_name`, `table\_name`, `status` ('pass'/'warn'/'fail'), and descriptive text.
- To monitor system health over the last 30 days: `SELECT check_name, status, COUNT(*) FROM dq_check_results WHERE checked_at >= NOW() - INTERVAL '30 days' GROUP BY check_name, status;`

Category 4: Scaling & System Evolution

Q13: How would you redesign this architecture if readings arrived every 10 seconds instead of every 3 hours?

Core Defense: *Transition from Batch Orchestration to Event-Driven Streaming.*

- Replace Airflow scheduled tasks with an event-driven ingestion service (e.g., Kafka producers polling or receiving webhooks).
- Kafka topics: `weather.readings.raw` and `aqi.readings.raw`.
- Stream Processing: Apache Flink or Spark Structured Streaming consuming from Kafka, computing 10-minute tumbling and sliding window aggregates, performing stateful deduplication via RocksDB state stores.
- Storage: Stream raw events to Iceberg/Delta Lake tables on S3/GCS and sink real-time aggregates to a time-series database (TimescaleDB or ClickHouse).

Q14: How would you deploy this to production in AWS or GCP?

Core Defense: *Cloud-native managed service mapping.*

- Orchestration: AWS MWAA (Managed Workflows for Apache Airflow) or GCP Cloud Composer.
- Raw Lake: Amazon S3 or Google Cloud Storage (replacing MongoDB with parquet objects in S3).
- Data Warehouse: Amazon Redshift, Snowflake, or Google BigQuery.
- Secrets: AWS Secrets Manager / GCP Secret Manager for the WAQI token.
- CI/CD: GitHub Actions running pytest, Docker container image builds pushed to ECR/GCR.

5. Advanced SQL Mastery & Code Defense

Data Engineering interviews almost always include a live SQL screening. Interviewers will test your understanding of window frames and streak detection. Here is the line-by-line mathematical explanation of your analytical queries:

Problem 1: The Gaps-and-Islands Math (`rn - rn2`)

Interview Question: 'Find all periods where a city experienced 3 or more consecutive readings of Unhealthy AQI (>150).'

The Trick: Calculate two independent row numbers over chronological order:

Reading TS	AQI	Is Unhealthy?	rn (All rows)	rn2 (Only Unhealthy)	Group Key (rn - rn2)
10:00 AM	80	False	1	-	-
01:00 PM	160	True	2	1	2 - 1 = 1 <-- Island 1
04:00 PM	175	True	3	2	3 - 2 = 1 <-- Island 1
07:00 PM	190	True	4	3	4 - 3 = 1 <-- Island 1 (Length 3!)
10:00 PM	95	False	5	-	- (Streak Broken)
01:00 AM	210	True	6	4	6 - 4 = 2 <-- Island 2

Why this works: As long as readings stay unhealthy, both `rn` and `rn2` increment by +1, keeping the difference `(rn - rn2)` constant. The moment a healthy reading occurs, `rn` increments but `rn2` does not, causing `(rn - rn2)` to shift to a new integer. Grouping by `(city\_id, island\_id)` allows you to run `COUNT(\*) >= 3` and calculate `MIN(reading\_ts)` and `MAX(reading\_ts)` for streak duration.

Problem 2: Rolling 24-Hour Average with `RANGE` Frame

Interview Question: 'Calculate the 24-hour rolling average temperature per city. Why use RANGE instead of ROWS?'

```
SELECT
  city_id, reading_ts, temperature_c,
  ROUND(AVG(temperature_c) OVER (
    PARTITION BY city_id
    ORDER BY reading_ts
    RANGE BETWEEN INTERVAL '24 hours' PRECEDING AND CURRENT ROW
  )::NUMERIC, 2) AS rolling_avg_24h
FROM fact_weather_reading;
```

Your Defense: ROWS BETWEEN 7 PRECEDING AND CURRENT ROW assumes exactly 8 observations at fixed 3-hour intervals. If a sensor was offline for 12 hours, a ROWS frame reaches back into observations from 36 hours ago to collect 8 rows, producing an inaccurate rolling average. RANGE BETWEEN INTERVAL '24 hours' PRECEDING operates on the *value* of the timestamp, including only rows whose timestamp is within the actual 24-hour clock window.

Problem 3: Ranking Functions Differences (`RANK` vs `DENSE\_RANK` vs `ROW\_NUMBER`)

Interview Question: 'How did you build the weekly city AQI leaderboard, and why use RANK()?'

- **`ROW\_NUMBER()`:** Arbitrarily breaks ties (1, 2, 3, 4). Inappropriate for a fair pollution ranking.
- **`DENSE\_RANK()`:** Tied cities share rank, next rank is sequential (1, 2, 2, 3).
- **`RANK()`:** Tied cities share rank, next rank skips by the number of ties (1, 2, 2, 4). We use `RANK()` because if two cities tie for 2nd place, the next city is legitimately the 4th most polluted city.

6. Common Interview Traps & What NEVER to Say

RED FLAG: Never say: 'I used Airflow XComs to pass all the weather data between tasks.'

STRONG ANSWER: Why it hurts: XComs are stored in the Airflow metadata database (Postgres/MySQL) as BLOBs/strings. Passing megabytes of data bloats the metadata DB and causes scheduler latency. Say instead: 'I used XComs only for lightweight task metadata (counts, run IDs, status flags). All heavy payloads were staged in MongoDB and retrieved by downstream workers.'

RED FLAG: Never say: 'The pipeline never fails because I put everything in a try-except block.'

STRONG ANSWER: Why it hurts: Silent exception swallowing is a critical bug in production data pipelines. Say instead: 'I caught specific network exceptions to handle partial failure gracefully, but logged errors to pipeline_run_log and raised explicit RuntimeError in the data quality gate to fail the task if data integrity was violated.'

RED FLAG: Never say: 'I just truncated and reloaded the tables on every run.'

STRONG ANSWER: Why it hurts: Truncate-and-load destroys historical trend analytics, prevents point-in-time rollups, and locks tables during reload. Say instead: 'I implemented an idempotent upsert pattern using PostgreSQL ON CONFLICT (city_id, reading_ts) DO UPDATE, preserving historical facts while permitting safe re-runs.'

RED FLAG: Never say: 'I tested the pipeline by watching it run in the Airflow UI.'

STRONG ANSWER: Why it hurts: Lacks software engineering maturity. Say instead: 'I developed an automated test suite of 77 pytest unit tests with mock cursors and JSON payloads for offline validation, plus a DagBag integrity test catching import/syntax bugs before deployment.'

7. Interview Day Quick-Fire Metrics Cheatsheet

Memorize these hard numbers and architectural primitives before your interview:

Metric / Primitive	Specification in This Project	Interview Talking Value
Active Target Cities	34 canonical Indian metropolitan cities	Demonstrates real multi-entity geographical tracking
Pipeline Cadence	Every 3 hours ($0 \div 3 \times 24$) = 8 runs/day	Trivial against API rate limits while keeping data fresh
Open-Meteo Optimization	Batched into 4 HTTP requests (10 cities/batch)	Shows awareness of socket overhead and URL limits
WAQI Extraction	34 dynamically mapped tasks (.expand())	Isolates per-station timeouts and failure blast radius
Data Quality Gate	10 automated checks (schema, range, nulls, freshness)	Fail-fast enforcement + audit persistence in single txn
Offline Test Suite	77 unit tests passing in < 2 seconds	Demonstrates test-driven development (TDD) discipline
Idempotency Key	(city_id, reading_ts) compound unique constraint	Guarantees re-runs and backfills never duplicate rows